Reveal-X

Secure Image Sharing using Visual Cryptography with AI-Based Tamper Detection

Inam Ullah (45521) · Zaka Khan (36503) · Muhammad Umar (44726)

Supervisor: Sir Yawar Abbas, Lecturer, Faculty of Computing

BS Cyber Security · Faculty of Computing
Riphah International University, Islamabad · December 2026

THE GAP

Two problems, and only one of them is solved today

— **Trust in the server.** Conventional messaging hands the relaying server a complete, viewable copy of every image. A compromise, an insider, or a disclosure order exposes it.

— **Integrity belongs to the channel, not the object.** TLS and at-rest encryption protect a file while it moves and while it sits. Once stored, no mainstream system can answer "has this specific image been altered, and where?"

— **Visual cryptography answers the first — but nothing else.** Published schemes since 1994 remove the need to trust the server. Not one of them carries an integrity check, and almost none has ever been put inside a working application.

Reveal-X addresses both, and measures how far each one actually goes.

IN ONE SLIDE

What Reveal-X does

Every shared image is split into two cryptographic shares. The server stores one; the other is delivered live and never written down. Both are then verified before the recipient can open anything.

2
shares per image
(either alone is a one-time pad)

0
bytes of Share 2
ever persisted

2
independent integrity
layers on every share

The invariant the whole design rests on: Share 2 is never written to server storage — and the test suite fails the build if it is.

SCOPE

Objectives: nine of ten delivered

#	Objective	Status
1–6	VC scheme, real-time messaging, XSS prevention, Argon2id, accessible UI, PWA	Delivered
7	Machine-learned tamper detection for shares	Delivered
8	End-to-end encryption of messages with AES-GCM	Delivered
9	Denoising CNN + ESRGAN enhancement	Not delivered — see slide 23
10	Evaluate accuracy, reconstruction fidelity and latency	Delivered

Objective 9 was dropped on evidence, not abandoned. The measurement that justifies the decision is in this deck.

Visual cryptography

What the scheme guarantees, and what it costs

A (2,2) XOR scheme, not classical stacking

```
share1    = os.urandom(image.size)        # OS CSPRNG, fresh every message
share2    = image XOR share1              # a one-time pad of the image
recovered = share1 XOR share2 = image     # exact, by associativity
```

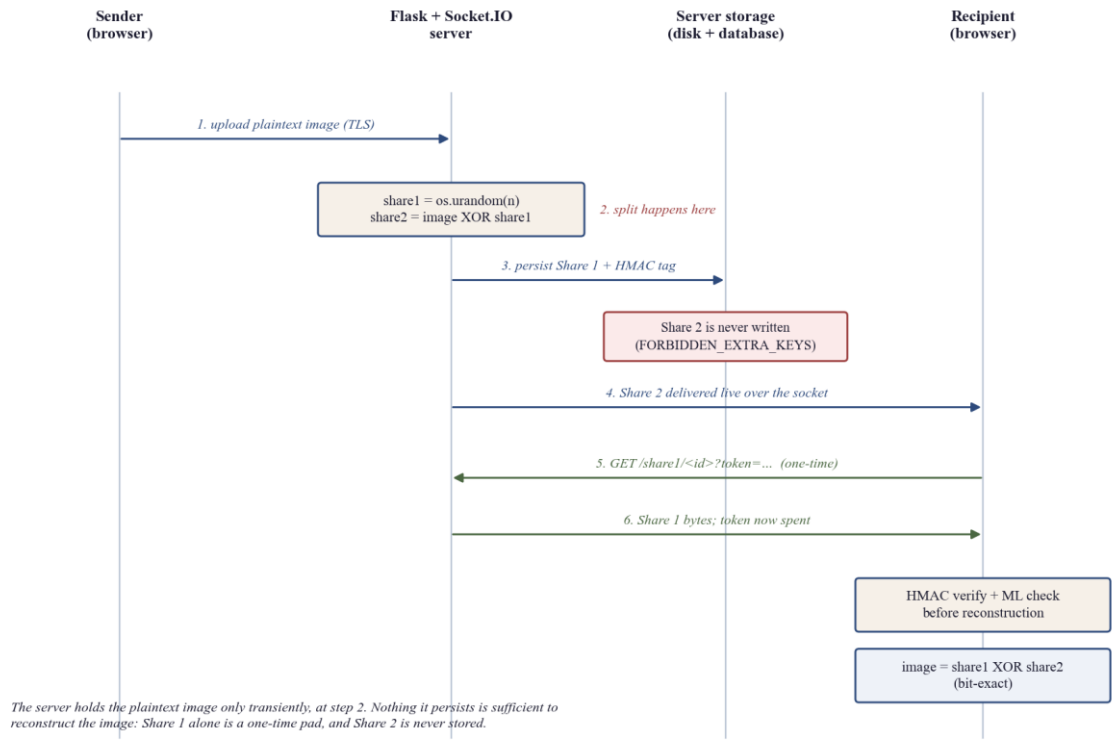
— **Why the OS CSPRNG?** A share from a seeded generator is reproducible by anyone who recovers the seed — that is obfuscation, not encryption.

— **Why XOR and not Naor–Shamir stacking?** Stacking decodes by eye and pays 2x–4x pixel expansion and half the contrast. We decode in software, so those costs would buy nothing.

Each Share 2 pixel is a one-time pad of the original — the construction Shannon proved perfectly secret.

DESIGN

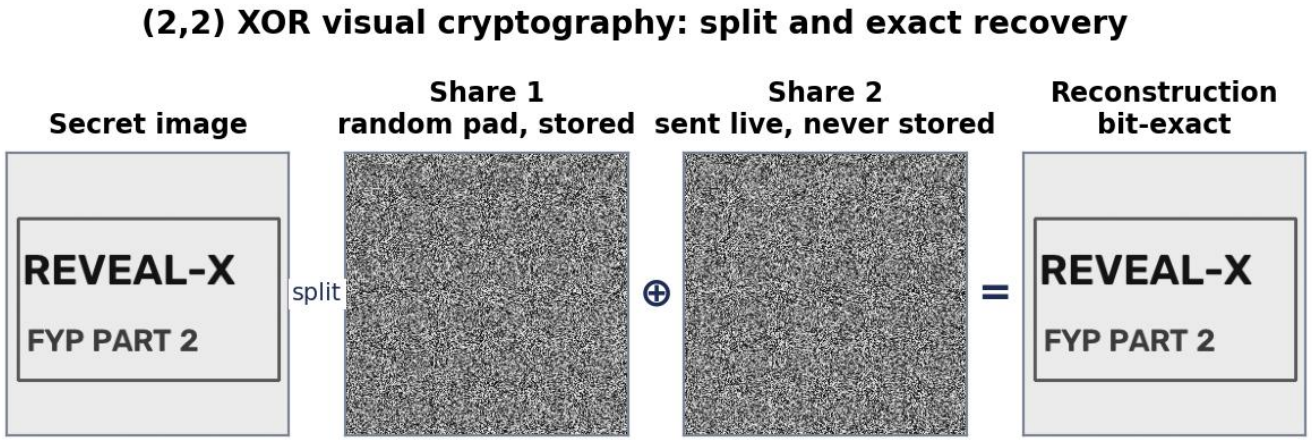
Where the split actually happens



The server sees the plaintext image only at step 2. Nothing it persists is enough to reconstruct it.

RESULT 1

Reconstruction is bit-exact



0.0

mean squared error

1.0000

structural similarity (SSIM)

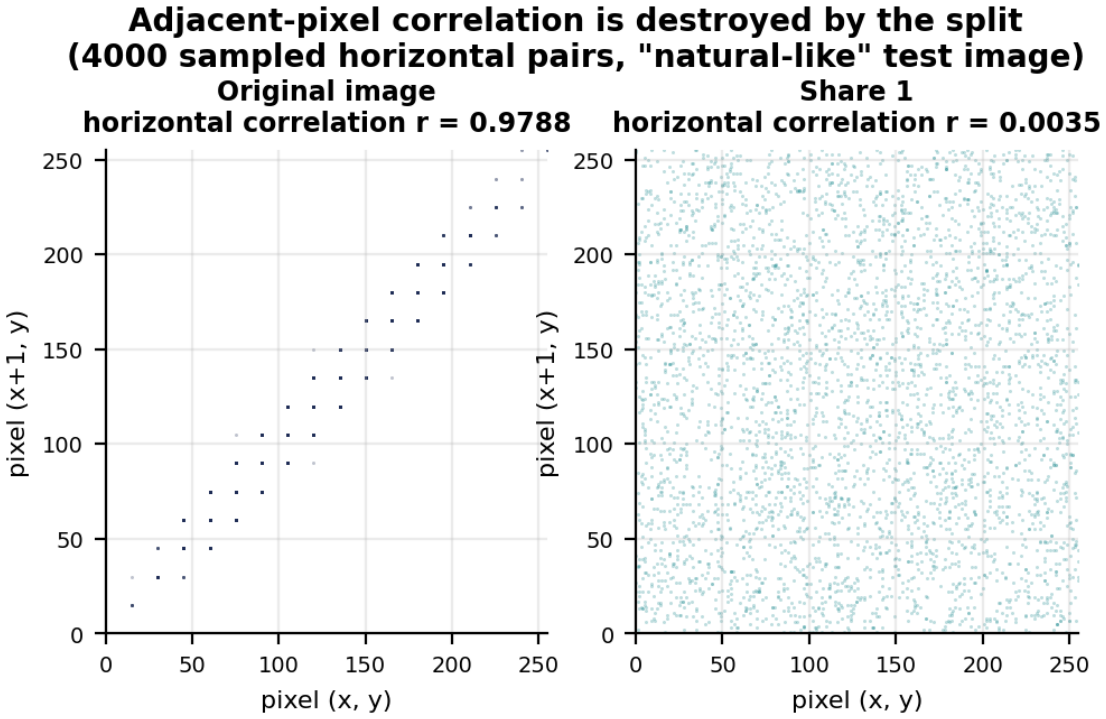
6 / 6

test images bit-identical

This corrected our own Part 1 claim of "measurable PSNR and SSIM degradation" — which is why Objective 9 lost its premise.

RESULT 2

A single share leaks nothing



Three independent tests

Shannon entropy

7.9973 / 8.000 bits per pixel

Adjacent-pixel correlation

original r = 0.9788

share r = 0.0035

Chi-square vs uniform

fails to reject for all 12

share arrays (p 0.117–0.764)

A share reaches the entropy maximum regardless of what it conceals — the structure an attacker would exploit is gone, not hidden.

RESULT 3

The cost nobody mentions: shares do not compress

Test image	PNG original	PNG Share 1	Original ratio	Share ratio	Storage overhead
gradient	461 B	65,976 B	142.2x	0.993x	143x
document	2,672 B	65,976 B	24.5x	0.993x	25x
natural-like	12,668 B	65,976 B	5.2x	0.993x	5x

— PNG is lossless entropy coding, so its output size bounds compressibility. A share compresses by 0.993x — the encoder finds no redundancy at all.

— Incompressibility is an empirical statement about entropy that depends on no statistical model. It is also why messages expire after one hour.

Tamper detection

What a classifier can see on random-looking data — and what it cannot

METHOD

The detector

— **13 statistical features, not raw pixels.** A valid share has a known profile: uniform histogram, zero neighbour correlation, maximum entropy, mean near 127.5. Every feature measures a departure from it.

— **RandomForest, 45 trees, depth 10.** 107.7 KB on the server; 40.2 KB exported to ONNX for the browser.

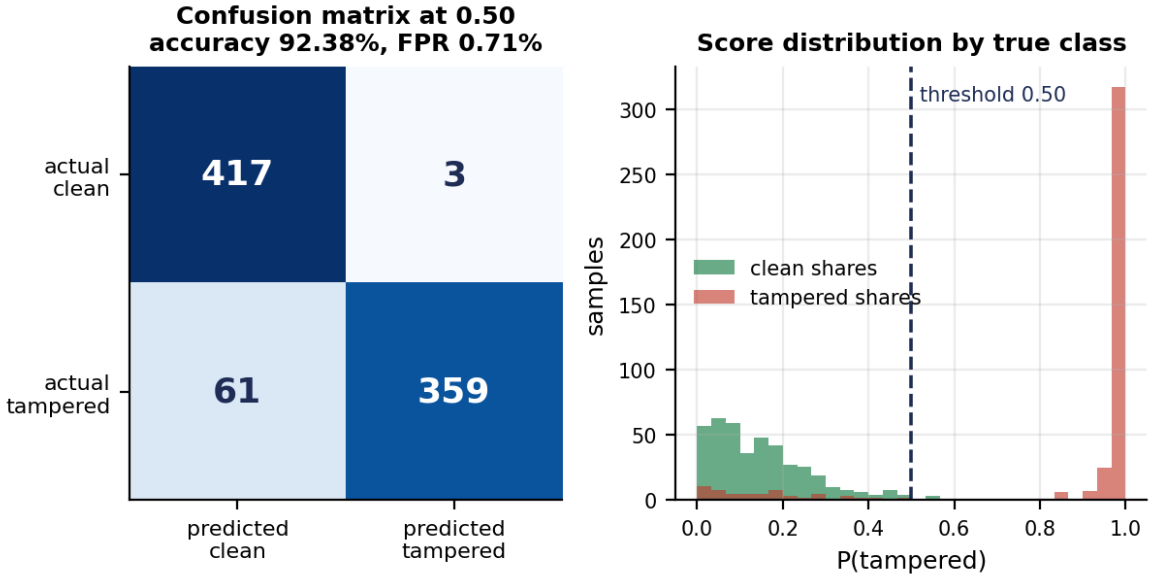
— **Trained on 240 paired samples, evaluated on 840 it never saw.** Every tampered sample is derived from the clean sample beside it, so the only difference between the classes is the attack itself.

— **Seven attack classes.** block, noise, blur, brightness, crop-and-rescale, scratch, JPEG re-encode — strength sampled from 0.18 to 0.55.

It runs twice: on the server, and again inside the recipient's browser — because after a one-time fetch the browser is the only party that can still check.

RESULT 4

Detection results on the independent set



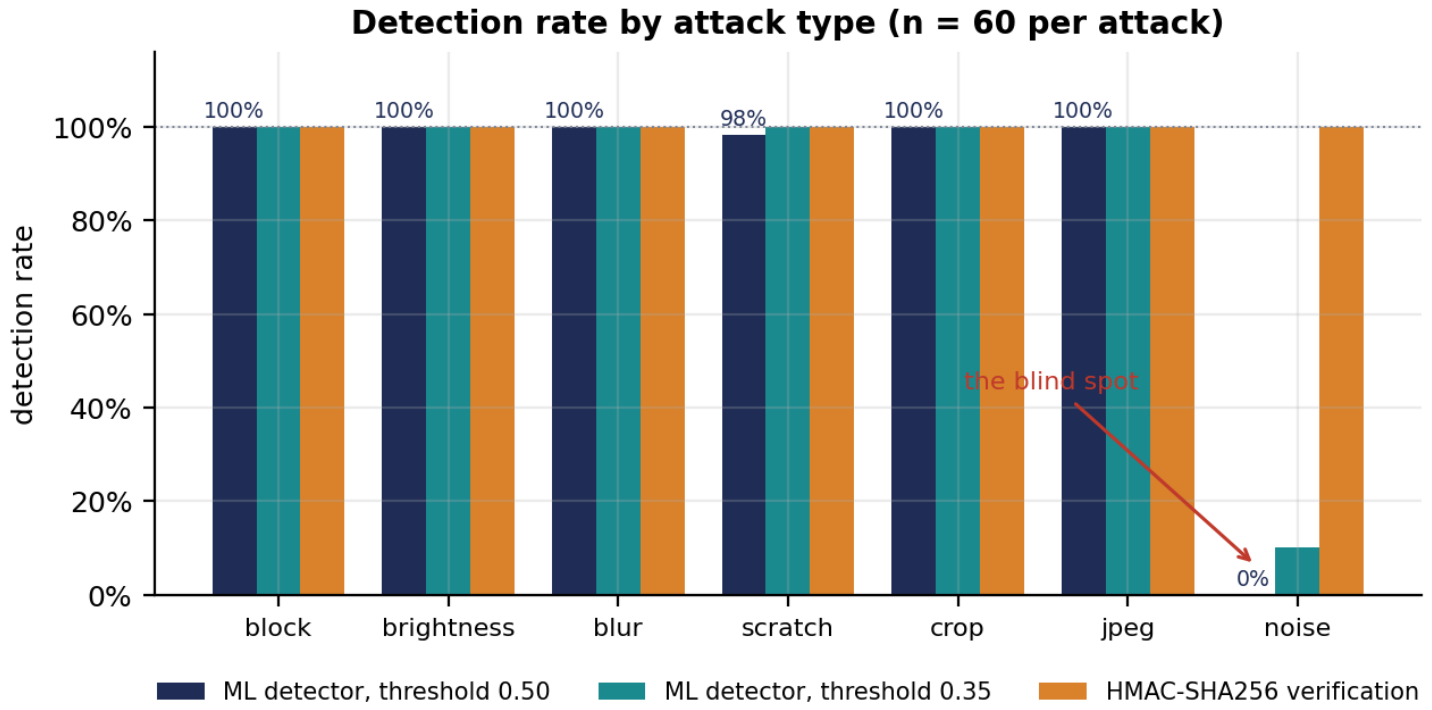
92.38%
accuracy (n = 840)

99.17%
precision

0.71%
false positive rate

RESULT 5

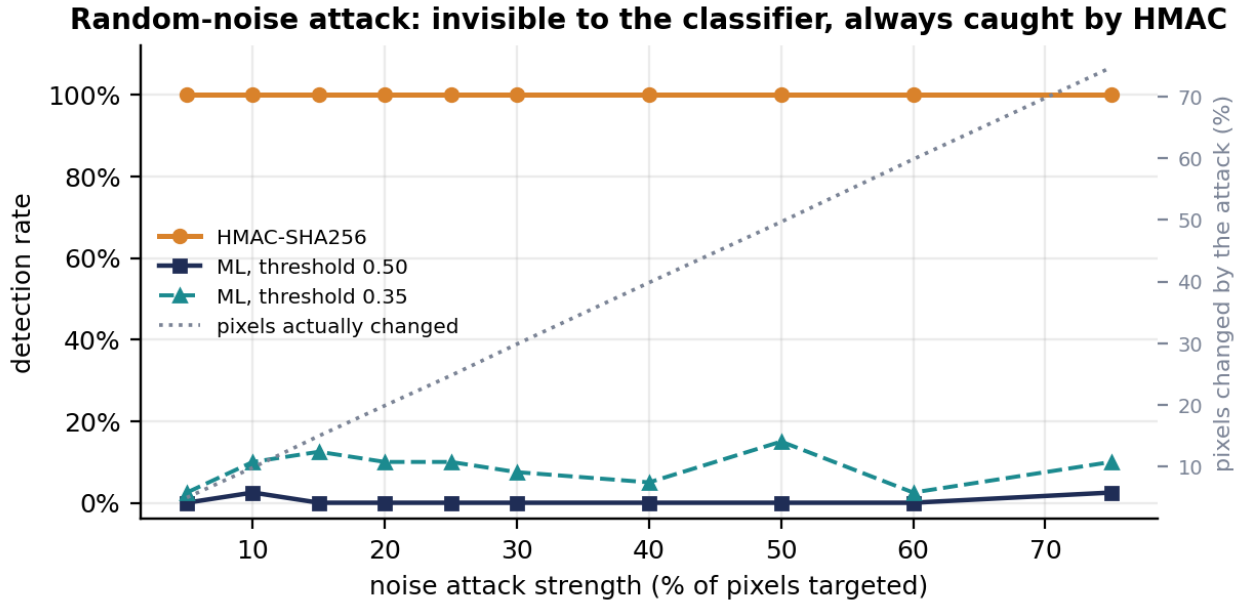
Six of seven attacks caught essentially without error



block, brightness, blur, crop and JPEG: 60/60 each. scratch: 59/60. noise: 0/60.

THE KEY FINDING

The blind spot — and why it is structural

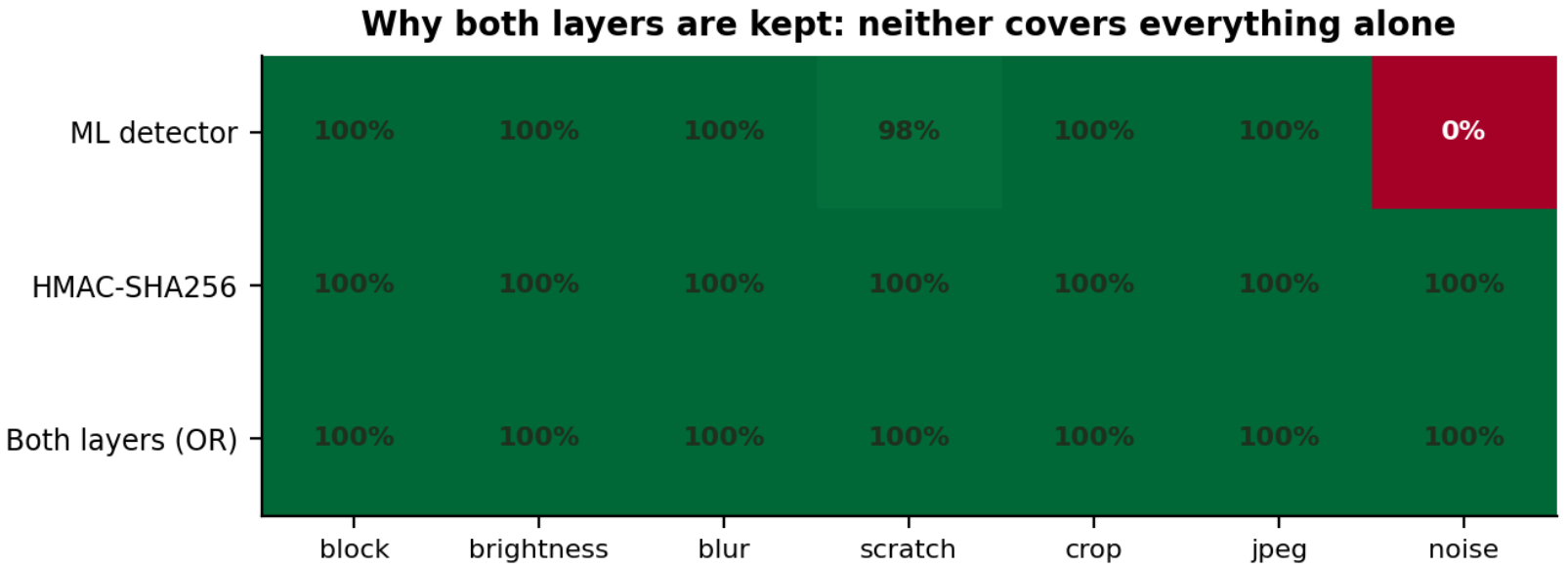


Swept from 5% to 75% strength, 40 trials each. Detection never exceeds 2.5%. The mean score stays at 0.137–0.176 — identical to the 0.149 assigned to genuinely clean shares.

The features measure departures from uniform randomness. This attack replaces uniform random pixels with fresh uniform random pixels. The result is still uniform random — so the feature vector is, by construction, that of a clean share.

RESULT 6

Why that is not fatal: two layers, different failures

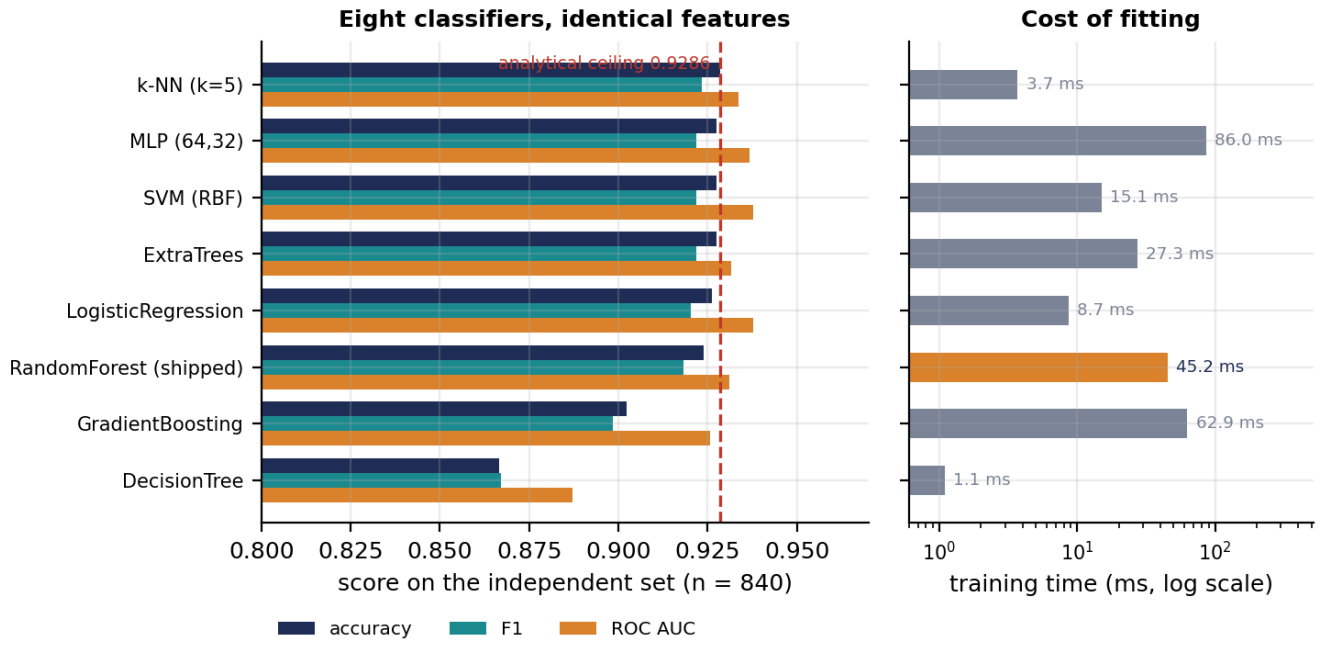


HMAC-SHA256 detected 420 of 420 tampered shares — including 60 of 60 noise attacks — and accepted 120 of 120 untouched shares.

HMAC is exact but says only that something changed, and a legitimate re-encode breaks it. The classifier survives re-encoding, localises damage to a 16x16 block and explains itself — but is blind to the one attack HMAC handles most easily. Neither layer is redundant.

RESULT 7

It is the features, not the learner



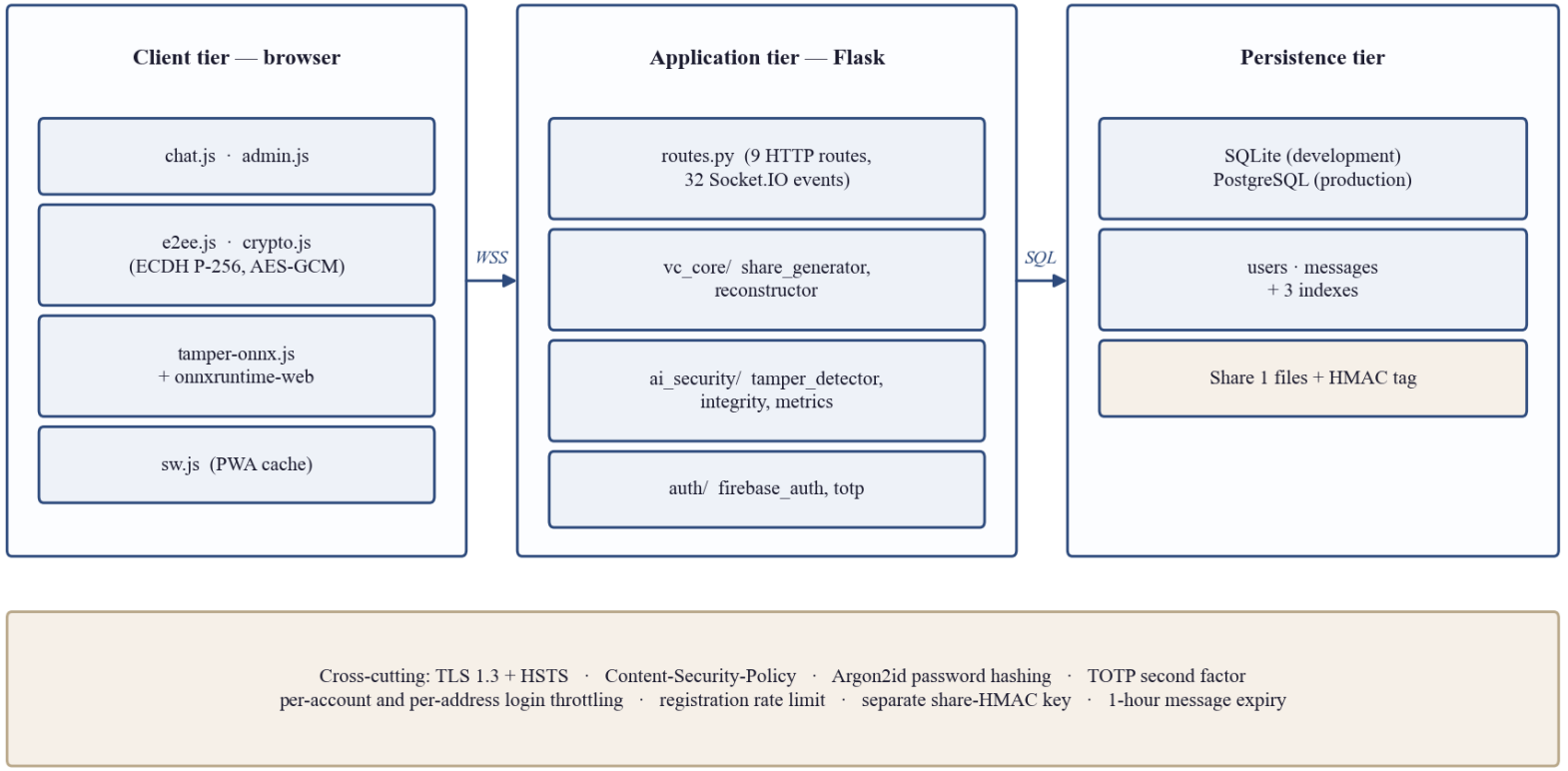
Eight classifier families on identical features: seven land between 0.9024 and 0.9286. Changing the model is not where effort should go.

System and engineering

Architecture, encryption, performance and verification

SYSTEM

Architecture



No GPU and no deep-learning framework anywhere in the dependency tree — the detector is a 40 KB ONNX file.

SECURITY

End-to-end encryption — and one honest boundary

— **ECDH P-256 → HKDF-SHA256 → AES-256-GCM.** Private keys are non-extractable CryptoKeys in IndexedDB — no script can read them out, including ours. A public-key fingerprint is shown for out-of-band comparison.

— **Argon2id, TOTP, three ranks, two throttle buckets.** Credentials are memory-hard hashed; the second factor is RFC 6238; an account may act only on one strictly below its own rank.

Images are the exception, and we say so

The browser uploads the image and the server performs the split, so the plaintext is in server memory at that moment. The guarantee for images is narrower: nothing the server persists is sufficient to reconstruct it. Moving the split into the browser is recommendation #1.

We also have no forward secrecy. A project claiming parity with Signal would be misrepresenting itself.

RESULT 8

Performance: the cryptography is not the expensive part

Image side	Share gen.	XOR recon.	HMAC	Features	Inference	Heatmap	Full check
128 px	0.006 ms	0.002 ms	0.010 ms	1.17 ms	1.80 ms	3.7 ms	7.2 ms
256 px	0.015 ms	0.003 ms	0.033 ms	4.69 ms	1.81 ms	13.2 ms	20.9 ms
500 px (cap)	0.050 ms	0.009 ms	0.120 ms	23.70 ms	1.81 ms	56.2 ms	85.8 ms

— At 256x256 the whole cryptographic core — both shares, the XOR and the HMAC — costs under a twentieth of a millisecond.

— Detection is the cost, and the heatmap is two thirds of it. A full check at the size cap is 86 ms, below the threshold a user perceives.

ENGINEERING

Verification

494
automated tests
(425 server, 69 browser)

0
failures
(2 environment-gated skips)

6,377
lines of test code

— **Tests run on every change.** Two Python versions, a container build, and a production-server smoke test — because gunicorn does not run on Windows and an unverified deployment command is only a note.

— **The tests themselves were mutation-tested.** Each security fix was deliberately reverted to confirm the new test fails. That found one test passing vacuously — it never reached the check it claimed to cover.

— **Negative results are recorded.** Three performance optimisations were tried and discarded on measurement; write-ahead logging made writes 60% slower here.

OBJECTIVE 9

What we did not deliver, and why

— **The premise was false.** Objective 9 assumed reconstruction was lossy and needed restoring. Measurement showed MSE of exactly 0 and SSIM of exactly 1.0000.

— **A denoiser would be trained on a degenerate objective.** DnCNN learns a mapping from degraded to clean. Here the input and the target are the same array, so the optimal learned mapping is the identity. There is nothing to remove.

— **Generative upscaling would be actively harmful.** ESRGAN synthesises detail that was not there. In a system whose purpose is to establish whether an image was altered, a hallucination is indistinguishable from tampering.

— **There is a defensible route, and we costed it.** Our attack simulator already produces matched damaged/clean pairs. A small residual CNN on those pairs is repair of a tampered share — not denoising of a clean reconstruction.

We judged it better to explain the omission than to ship a solution without a problem.

What we would do differently

— **Split in the browser.** The one change that would convert a partial image guarantee into a full one and remove the server from the trusted base entirely.

— **Reconcile documentation instead of appending to it.** Our Part 1 chapters asserted a client-side split, scrypt hashing and a demonstration-only key exchange. The error survived a revision because new results were appended rather than reconciled.

— **Get independent security testing.** Our own review found and fixed three real issues — but two of them were in controls we had written and believed correct.

— **Evaluate against an adaptive adversary.** Nothing here bounds classifier performance against an attacker who knows the thirteen features. The noise attack is an accidental instance of exactly that.

Conclusion

— Information-theoretically secure image sharing works inside a real-time chat application, at a cryptographic cost measured in tens of microseconds.

— The contribution is the integrity layer above an established scheme: a keyed tag and a learned detector that provably cover different failures.

— The clearest single result is a negative one — a classifier over departures from randomness cannot detect an attack that injects randomness, and no learner over those features can.

— That is precisely why the cryptographic layer is not optional.

Nine of ten objectives delivered. 94-page report, 494 automated tests, every figure reproducible from the repository.

Thank you

Questions

Inam Ullah · Zaka Khan · Muhammad Umar

github.com/1337PAIN/Reveal-X-chat